

Phase 1 “ ” “ ”

- test_planner.launch.py experiment:=p0_open_sky
- export src/iap/results/planner_validation/exports/...
- manifest experimentscenarioplanner_safety_profilep0.enable_risk_grid
- rosvag /planning/risk_grid_health
- P0 RiskGridMapP0RuntimeIntegritySnapshotFuturePLPredictor

- /planning/risk_grid_health p0_risk_grid_health.csv
- Phase 1 heatmap / trajectory equivalence
- P0-5 manual synthetic field P0-6 obstacle-overlap field GTest launch

Phase 1

Phase 1

4 launch

1	P0-1 open sky	experiment:=p0_open_sky	P0 health
2	P0-2 degraded GNSS + good LiDAR	experiment:=p0_open_sky scenario:=gnss_degraded_lidar_good	P0
3	P0-3 corridor degeneracy	experiment:=p0_open_sky scenario:=lidar_corridor_degenerate	P0 health / risk field
4	P0-4 fallback only	experiment:=p0_open_sky scenario:=fallback_only	unknown/fallback

P0-5P0-6 launch GTest

```
cd /home/dev/ws_iap
source install/setup.bash
```

00 launch 0000000000 build0

```
cd /home/dev/ws_iap
colcon build --packages-select iap ego_planner --symlink-install
source install/setup.bash
```

00 experiment 00000000

```
ros2 launch iap test_planner.launch.py --show-args | grep experiment
```

0 0 0000 P0 0000

```
cd /home/dev/ws_iap
source install/setup.bash

./build/iap/test_risk_grid_map
./build/ego_planner/test_p0_risk_grid_runtime
./build/iap/test_integrity_snapshot
./build/iap/test_future_pl_field_predictor
```

00000000000000 P0 00000unknown/stale0runtime 000snapshot 000

0 1 000000P0 open sky

00000000 Phase 1 000000000

```
cd /home/dev/ws_iap
source install/setup.bash

ros2 launch iap test_planner.launch.py \
  experiment:=p0_open_sky \
  run_duration_s:=90 \
  validation_duration_s:=85 \
  start_rviz:=true \
  run_validator:=true \
  record_bag:=true
```

00000000 export0

```
cd /home/dev/ws_iap

RUN=$(ls -td
```

```
echo "$RUN"
python3 -m json.tool "$RUN/test_planner_manifest.json"
python3 -m json.tool "$RUN/test_planner_validation_summary.json"
```

```
experiment = p0_open_sky
scenario = gnss_open_sky
planner_safety_profile = off
p0.enable_risk_grid = true
planner_enable_p1 = false
planner_enable_p2 = false
planner_enable_p3_local = false
planner_enable_p3_global = false
planner_enable_p4 = false
planner_enable_p5_runtime = false
planner_enable_p5_final = false
```

bag P0 health

PROF

```
/planning/risk_grid_health
```

☐ ☐ A ☐

```
cd /home/dev/ws_iap
source install/setup.bash

BAG=$(ls -td
src/iap/results/planner_validation/bags/test_planner_p0_open_sky_gnss_op
en_sky_* | head -1)
ros2 bag play "$BAG" --clock
```

□□ B□

```
cd /home/dev/ws_iap
source install/setup.bash

ros2 topic echo /planning/risk_grid_health --once
```

□□□□ std_msgs/String□□□□ JSON□□□□□□

```
{
  "ready": true,
  "stale": false,
  "age_s": ...,
  "valid_ratio": ...,
  "unknown_ratio": ...,
  "generation_id": ...,
  "reason": "..."
```

P0-1 □□□

—
PROF

- ready □□□□ true
- stale □□□□□□□□ false
- generation_id □□□□
- valid_ratio □□□□□□□□□□□□ > 0.6
- unknown_ratio □□□□□
- validator summary □ passed □□□ true

□ 2 □□□□ **degraded GNSS + LiDAR good**

```
cd /home/dev/ws_iap
source install/setup.bash

ros2 launch iap test_planner.launch.py \
  experiment:=p0_open_sky \
  scenario:=gnss_degraded_lidar_good \
  run_duration_s:=90 \
```

```
validation_duration_s:=85 \
start_rviz:=false \
run_validator:=true \
record_bag:=true
```

□□□

```
RUN=$(ls -td
src/iap/results/planner_validation/exports/test_planner_p0_open_sky_gnss
_degraded_lidar_good_* | head -1)

echo "$RUN"
python3 -m json.tool "$RUN/test_planner_manifest.json"
python3 -m json.tool "$RUN/test_planner_validation_summary.json"
```

□□□□

- scenario □□□ gnss_degraded_lidar_good
- p0.enable_risk_grid □□□ true
- planner_safety_profile □□□□□ off
- validator □□□□

□□□□□□□□□□□□□□□□ P1-P5 □□□□□□□ P0 □ degraded + LiDAR good □□□□□□□□□□ risk grid□

□ 3 □□□□corridor degeneracy

```
cd /home/dev/ws_iap
source install/setup.bash

ros2 launch iap test_planner.launch.py \
  experiment:=p0_open_sky \
  scenario:=lidar_corridor_degenerate \
  run_duration_s:=90 \
  validation_duration_s:=85 \
  start_rviz:=false \
  run_validator:=true \
  record_bag:=true
```

□□□

```
RUN=$(ls -td
src/iap/results/planner_validation/exports/test_planner_p0_open_sky_lida
r_corridor_degenerate_* | head -1)

echo "$RUN"
```

```
python3 -m json.tool "$RUN/test_planner_manifest.json"
python3 -m json.tool "$RUN/test_planner_validation_summary.json"
```

□□□□

- P0 0000
- P1-P5 0000
- validator 0000
- /planning/risk_grid_health 000000

Phase 2 / P5 corridor P0-only hard gate

4 fallback only

```
cd /home/dev/ws_iap
source install/setup.bash

ros2 launch iap test_planner.launch.py \
  experiment:=p0_open_sky \
  scenario:=fallback_only \
  run_duration_s:=90 \
  validation_duration_s:=85 \
  start_rviz:=false \
  run_validator:=true \
  record_bag:=true
```

111

```
RUN=$(ls -td
src/iap/results/planner_validation/exports/test_planner_p0_open_sky_fall
back_only_* | head -1)

echo "$RUN"
python3 -m json.tool "$RUN/test_planner_manifest.json"
python3 -m json.tool "$RUN/test_planner_validation_summary.json"
```

```
echo "$RUN"
python3 -m json.tool "$RUN/test_planner_manifest.json"
python3 -m json.tool "$RUN/test_planner_validation_summary.json"
```

□ □ □ □ □ □ □ □ □

- P0 `unavailable/fallback` `0` risk
- `health` `explicit reason`
- `unknown_ratio` `0.0000000000`
- `ready=false` `reason 000`

□□□□□□□□/□□□ **4** □□

□ □ □ □ □ □ □ □

1. launch []
2. export dir []
3. bag dir []
4. manifest + validation_summary []

□□□□□□

```
manifest:
  p0.enable_risk_grid = true
  planner_safety_profile = off
  P1-P5 switches = false

summary:
  passed = true

bag:
  /planning/risk_grid_health exists

P0 health:
  reason explicit
  no silent zero-risk fallback
  generation_id refreshes when ready
```

□□□□□□

□□□□□□□□□□

```
ros2 launch iap test_planner.launch.py \
  experiment:=p0_open_sky \
  run_duration_s:=90 \
  validation_duration_s:=85 \
  start_rviz:=true \
  run_validator:=true \
  record_bag:=true
```

□□□□□□ export □□□□□□□□□□ P0-1 □□□□□□□□□□□□ P0-2□